Security self-assessment

Praetor lets an AI agent move a user's money, so the design question is not "is the agent trustworthy" (it is not) but "what is the worst the agent, or anything around it, can do". This document answers it with evidence from the code as built: the permission tiers and the contract function that enforces each, the nineteen invariants with the tests that prove them, the threat model, the known risks, and the results of the tools. Chinese version: docs/zh-CN/SELF_ASSESSMENT.md. Architecture: docs/ARCHITECTURE.md.

Snapshot: 2026-10-18, refreshed after D44 to D75, after the security review of F10, web push and the sandbox (D80 to D83), after the red team, the M18 rehearsal and the Pyth feed (D86 to D90), and after the teammates' phone-test fixes (D91, D93 to D95) and the M16 live run (D92, D96). Built since the M17 version: the web app with the consortium console and the interim sandbox (M13), the mini program (M14), agent key rotation and owner recovery (F10) and the Guomi smoke (M15), the Base Sepolia deployment, the live-run scripts and the red-team suite (M16), the production compose override and its refusal check (M18). The red team ran against the local stack on 2026-10-18 with 0 attacks executed (section 9); M18 passed on the fork (D90); on 2026-10-18 the team passed the mini program's link, SOTER Face ID approval, replay refusal, freeze, unlink and Chinese UI on an iPhone (risk 10). Every Phase B milestone gate is ticked in `docs/PROGRESS.md`: M16 closed on 2026-10-18 with the evidence run on Base Sepolia (Pyth prices, the real model, WeCross; `docs/EVIDENCE.md`, D96), whose on-chain refusals include a prompt injection to a sanctioned address (SANCTIONED), a new payee before the owner's approval (UNKNOWN_PAYEE) and a payment while frozen (FROZEN). Still to come, in Phase C: the TEEs on Phala Cloud (attestations still come from the dstack simulator), the cloud deployment, the pentest (section 10), the held-out red-team cases and the I19 scan (not built).

1. Evidence at a glance

Check	Result	Where
Foundry, Base contracts	235 unit, fuzz and invariant tests pass (quick profile), plus 13 fork tests on the real Safe7579 (`make test-fork`, needs network)	`contracts/base/test`
Foundry, FISCO contracts	30 tests, plus 8 for the Guomi build of the boards (`FOUNDRY_PROFILE=sm`, D55)	`contracts/fisco/test`, `contracts/fisco/sm/test`
Fuzz and invariants	10,000 runs per fuzz test and 256 x 64 invariant runs in the milestone profile (`make test-contracts-full`); 256 and 32 x 32 on every commit	`contracts/base/foundry.toml`
Line coverage, `MandateHook.sol`	**99.52 % (207 / 208)**; hook with its four libraries 99.5 % (821 / 825); target 90 %	section 7
Slither, Praetor's Base code	High 0, Medium 27 (all explained), Low 52	slither.md
Slither, vendored WeCross proxy	High 3. Medium 8, all triaged as not exploitable; filtered like `lib/` since D49, so **`make slither` passes** (it also analyses the FISCO contracts)	slither.md
`pnpm audit --prod`	4 high, 7 moderate, 2 low; each assessed, none upgraded yet	deps.md
gitleaks (full history)	no leaks; the history scan also works from a git worktree since `scripts/checks/gitleaks.sh` mounts the common git directory	deps.md

Check	Result	Where
Integration and e2e (flows F1 to F10)	F1 to F9 green at the gates of M3 to M11 (`docs/PROGRESS.md`). M12 gate: integration 16/16 files and e2e 41/41 on a fresh fork with WeCross as the transport (`BRIDGE=wecross`). M13 gate: the web app's Playwright journey F1 to F10 with the sandbox spec (`apps/web/e2e`) 15/15, 34 screenshots each with axe. M15 gate: F10 (`test/e2e/F10-rotation-recovery.test.ts`) 3/3 and the 48 h execution (`test/warp/F10-recovery-execute.test.ts`), on fresh forks. These are the gate results recorded in `docs/PROGRESS.md` ; not re-run for this document (the shared local stack was in use)	`test/integration` , `test/e2e` , `test/warp` , `apps/web/e2e`
Production refusals (I13)	`scripts/deploy/prod-refusals.sh` starts the bank, agent-signer, soter-attester, agent-runner and the api with `PRAETOR_ENV=prod` and their test-only settings (simulator attestations, the fake Tencent, the fake LLM), and expects each to refuse to start; run by `make rehearsal` : all five refused in the M18 rehearsal on 2026-10-18, beside the smoke F1, F5, F8 16/16 on `compose.prod.yml` (D90)	`scripts/deploy` , `test/infra/compose-prod.test.ts`
Red team ("N attacks, 0 executed")	Local stack, 2026-10-18: scripted **85 attacks, 0 executed**; global (shared state) **3 attacks, 0 executed**; real model **12 attacks, 0 executed** (3 blocked by the stack, 9 declined by the model). Findings of the earlier runs fixed and re-tested (D86 to D88)	section 9, redteam.md
Live evidence run, Base Sepolia (M16)	2026-10-18 (D96): C0 to F8 on one account at `DEMO_SCALE=100` with Pyth prices, the real model (DeepSeek through OpenCode) and WeCross; 21 BaseScan links to Base transactions, among them Refused SANCTIONED (a prompt injection, F5b), Refused UNKNOWN_PAYEE then ApprovalRegistered and ApprovalConsumed with Decision OK (F6), StatusApplied through WeCross then Refused FROZEN (F9), and audit anchors (F8)	EVIDENCE.md
Pentest	**Phase C**, report in `docs/security/pentest.md`	section 10
TEE deployment checklist	items enforced in code today are listed; results **after deployment (Phase C)**	tee-checklist.md

Reproduce: `make test` (guards, gitleaks, type-check, every unit test, Foundry quick profile), `make test-contracts-full` , `make test-fork` , `make slither` , `pnpm audit --prod` , and with the local stack up (`make up`) `make test-int` and `make e2e` .

2. Permission tiers

Every execution of a Praetor account passes `MandateHook.preCheck` (I1), whoever signed it. The hook's single evaluator, `DecisionLib.evaluate` , returns for each call a reason and a tier; `check()` and `checkBatch()` expose the same evaluator as views, so a refusal recorded by `RefusalLog` always carries the reason the hook computed (I11). Rule of thumb everywhere: **stricter needs fewer signatures than looser.**

Tier	Example action	Verifying contract function	Signer(s)	Proven by
T0 Never	The agent, prompt-injected, pays 5,000 USDC to a sanctioned address; any call while FROZEN; over the bank caps or the hard cap; delegatecall, ETH value, unknown target or selector; stale price; slippage	`MandateHook.preCheck` reverts `MandateRefused(reason, callHash)` ; `RefusalLog.log` records the reason from `check()`	nobody: no approval lifts a T0 reason	`contracts/base/test/unit/MandateHookDecide.t.sol` (`test_A2_sanctionedPayee` , `test_A5_overHardCapPolicyOrBank` , `test_A6_bankDailyCapIsT0EvenWithApproval` , `test_approvalNeverLiftsFrozen` , `test_rule5_hiddenCalls`); `test/invariant/HookInvariants.t.sol` (`invariant_I8_bankCapsNeverExceeded`); `test/e2e/F5-agent-payment.test.ts` ("4-5: an injected 5,000 USDC payment to a sanctioned address is refused SANCTIONED")

Tier	Example action	Verifying contract function	Signer(s)	Proven by
T1 Autonomous	"Pay the café 12.50" to an allowlisted payee under the auto cap; a rebalance inside the bands; a due standing order	`AgentKeyValidator.validateUserOp` (agent key, `execute` only), then `MandateHook.preCheck`; standing orders: `StandingOrders.execute` then `executeFromExecutor`, checked by the hook's executor path	the agent key in the agent-signer TEE; a standing order is triggered by anyone	`MandateHookDecide.t.sol` (`test_baselinePaymentIsAutonomous`, `test_rebalanceInsideTheBandsIsAutonomous`); `MandateHookExecute.t.sol` (`test_allowedPaymentRecordsSpendAndEmitsDecision`, `test_executorPathSkipsTheT2GroupButNotT0`); `test/fork/Safe7579Fork.t.sol` (`test_agentPaysAutonomouslyAndARefusalIsLoggedWithTheHooksReason`, `test_standingOrderPaysThroughExecuteFromExecutor`); e2e F4, F5 ("2:"), F7
T2 Human	Pay Café Lumen 300 (unknown payee, over the auto cap), or over the user's budget, count, buffer or rebalance limit	`MandateHook.registerApproval(Tier2Approval, OwnerAuth)` registers an approval bound to one `callHash`; the next `preCheck` consumes it	the owner: a passkey verified on-chain with WebAuthn.sol, or a `SoterApproval` signed by an active attester TEE for the account's WeChat binding	`MandateHookOwner.t.sol` (`test_passkeyApprovalRegisters`, `test_soterApprovalNeedsPolicyBindingAndActiveAttester`, `test_soterCounterAndBinding`); `MandateHookDecide.t.sol` (`test_T2ApprovalLiftsAndIsBoundToTheCall`, `test_oneApprovalCoversOneCallInABatch`); `MandateHookExecute.t.sol` (`test_approvalIsConsumedOnce`); `test/e2e/F6-approval.test.ts` (passkey, SOTER, replayed counter)
T3 Owner + bank	Set or change the mandate (`SET_POLICY`), add a standing order, link WeChat, set or rotate the agent key, add an owner passkey (a new device), install or remove a module	`MandateHook.applyOwnerAction(OwnerAction, payload, OwnerAuth, bankSigs)`; module changes then need `execute(self.installModule / uninstallModule)` with the registered `moduleApprovals` entry (D8)	owner passkey, or SOTER for `SET_POLICY`, `ADD_STANDING_ORDER`, `SET_AGENT_KEY` and `MODULE_CHANGE` (`LINK_WECHAT` and `SET_OWNER_KEY` are passkey only), **and** the bank's ECDSA signature; the bank co-signs only after its own rule checks (D26)	`MandateHookOwner.t.sol` (`test_setPolicyNeedsOwnerAndBank`, `test_setPolicyWithSoterAndBank`, `test_standingOrderAddNeedsBankCancelAlone`, `test_setAgentKeyOnlyTheBankVerifiedKey`, `test_setOwnerKeyPasskeyAndBankOnly`, `test_linkWechatIsPasskeyOnlyAndChecksTheProof`, `test_moduleChangeRegistersAModuleApproval`); `Safe7579Fork.t.sol` (`test_I1_approvedHookRemovalSucceeds`); `services/bank/test/cosign.test.ts`; `services/api/test/devices.test.ts` ("registers a second device for the signed-in owner and returns the key for SET_OWNER_KEY; it signs in once Base holds it"); `services/api/test/rotation.test.ts` ("creates the key version after the one Base holds for the account"); e2e F1 ("14: SET_AGENT_KEY"), F2, F3; F10 ("3-5: rotate: ...")
T3- Owner alone	Revoke the agent key, pause the agent (and resume a pause the owner set), remove a payee or merchant, cancel a standing order, unlink WeChat, cancel a pending recovery	`MandateHook.applyOwnerAction` without bank signatures	owner passkey or SOTER	`MandateHookOwner.t.sol` (`test_pauseAndResumeAreOwnerAlone`, `test_removePayeeAndMerchantUpdatesThePolicyHash`, `test_linkWechatWithoutPolicyThenUnlinkAlone`, `test_ownerCancelsRecoveryAlone`); the bank refuses to co-sign these kinds (`cosign.test.ts`); `services/api/test/mp-routes.test.ts` ("prepares the action over the binding on Base, and its success ends the mini program session"); e2e F10 ("1-2: stop my agent: REVOKE_AGENT_KEY by the owner alone; the key no longer validates; its AgentMandate is revoked")

Tier	Example action	Verifying contract function	Signer(s)	Proven by
Recovery	The owner lost every passkey: after its own re-KYC matched the sealed PersonKYC record, the bank schedules the new passkey (D47)	`MandateHook.scheduleRecovery` (bank only, `SET_OWNER_KEY` only), executable by anyone after 48 h through `executeRecovery`, which replaces every owner key, clears the WeChat binding, increments the account epoch and revokes the agent key; the account is RESTRICTED meanwhile and any remaining passkey cancels alone	the bank (its Base signer key; the transaction is sent by its desk account)	`MandateHookOwner.t.sol` (`test_recoveryScheduledByBankExecutesAfter48h`, `test_ownerCancelsRecoveryAlone`, `test_recoveryOnlyForSetOwnerKey`); `services/bank/test/recovery.test.ts` ("signs SET_OWNER_KEY for the new passkey with the bank key and schedules it on Base; tells the user itself"); `services/api/test/devices.test.ts` ("registers the new passkey for the email, then starts the bank's re-KYC with it; status and execution are public"); e2e F10 ("6: recovery: a new passkey for the email and a matching re-KYC schedule it (RESTRICTED); a mismatching one does not; a remaining passkey cancels") and `test/warp/F10-recovery-execute.test.ts` ("6: scheduled after the matching re-KYC; 48 h later executeRecovery: only the new passkey signs in, the agent key is revoked")
T4 Consortium	Regulator freeze (1 of 3), loosen back to ACTIVE (2 of 3), heartbeat (any 1), sanctions list (1) and delist (2), jurisdiction block (1) and unblock (2), asset suspension (any 1), caps, KYA, asset activation and attester activation (the bank), attester deactivation (any 1), audit anchors (any 2)	`MandateHook.applyStatus / applyHeartbeat / applySanctions / applyJurisdiction / applyAsset / applyCaps / applyKYA / applyAttester`, `Anchor.anchor`: `ecrecover` of distinct, ascending member signers, `nonce == stored + 1` (allocated by FISCO), `block.timestamp <= deadline`	member Base signer keys (bank, regulator, operator)	`MandateHookConsortium.t.sol` (22 tests, e.g. `test_oneMemberFreezesAnAccount`, `test_looseningNeedsTwoMembers`, `test_sameStatusCountsAsLoosening`, `test_sanctionsListOneDelistTwo`, `test_jurisdictionBlockOneUnblockTwo`, `test_assetActivationNeedsBankSuspensionAnyMember`, `test_attesterActivateBankDeactivateAny`, `test_capsNeedTheBank`, `test_kyaNeedsTheBank`); `test/fuzz/HookFuzz.t.sol` (`testFuzz_I3_signerSets`, `testFuzz_I18_nonce`); `Anchor.t.sol`; `contracts/fisco/test/Board.t.sol`, `Boards.t.sol`; `test/e2e/F9-governance.test.ts` (8 tests)

Exact thresholds as built (`contracts/base/src/hook/ConsortiumLib.sol`): a status change is "tightening" only when the new status is stricter than the stored one; the same or a looser status needs 2 signatures. Asset changes count as loosening when the status becomes less strict, the bank's max weight rises, or the kind changes. A heartbeat is accepted with any one member and at most one day of validity (`MAX_HEARTBEAT_TTL`). `Anchor` accepts any two members; that the bank is one of them is the operator service's policy (D41), not a contract rule.

Reason codes as built: the hook returns the reasons of spec 02 in the decision order of spec 03. Two codes of the frozen protocol are never returned by `check()`: `TIER2_REQUIRED` (14) and `TIER2_INVALID` (15). An invalid approval does not produce a refusal reason: `registerApproval` and `applyOwnerAction` revert with a custom error (`BadOwnerAuth`, `BadSoter`, `DigestUsed`, `BadExpiry`, `SlotBusy`, `SoterNotEnabled`, `BankRequired`), so nothing is registered and the payment stays refused with its original T2 reason (D65).

3. Invariants and the tests that prove them

Status: **Proven** = automated tests in `make test`; **Fork** = Foundry fork tests on the real Safe7579 (`make test-fork`, network); **Gate** = integration or e2e tests run at milestone gates against the local stack; **Partial** = proven for part of the statement, the rest stated; **Not yet** = no automated proof.

#	Invariant	Status	Proven by
I1	Every execution passes `MandateHook`; Safe native owners cannot execute	Fork, Proven, Gate	`Safe7579Fork.t.sol`: `test_I1_execTransactionAlwaysReverts`, `test_I1_rawUninstallRefusedByThePasskeyValidator`, `test_I1_uninstallThroughExecuteWithoutApprovalRefused`, `test_I1_approvedHookRemovalSucceeds`, `test_freshAccountRefusesEverything`. `MandateHookExecute.t.sol`: `test_moduleChangeNeedsAnApproval`, `test_innerModuleCallWithoutOuterApprovalRefused`, `test_executionModesOtherThanSingleAndBatchAreHidden`, `test_executorPathOnlyFromStandingOrders`. `NoOwner.t.sol`: `testFuzz_neverSigns`. `PasskeyValidator.t.sol`: `test_refusesRawModuleCalls`. `test/integration/bundler-base.test.ts`: the two "I1:" tests
I2	Default deny: any target or selector not modelled is refused	Proven	`HookFuzz.t.sol`: `testFuzz_I2_defaultDeny`, `testFuzz_I2_unknownSelectorsOnKnownContracts`, `testFuzz_nonCanonicalBatches`. `MandateHookDecide.t.sol`: `test_unknownTargetRefused`, `test_usdcSelectorsOutsideThePatternRefused`, `test_nonCanonicalTransferEncodingsRefused`, `test_metalTransfersAndOtherTokenSelectorsRefused`. `MandateHookExecute.t.sol`: `test_otherSelectorsValueAndShortDataAreHidden`, `test_malformedExecuteEncodingsAreHidden`, `test_nonCanonicalBatchIsHidden`, `test_selfCallInsideABatchIsHidden`
I3	Consortium messages need valid member signatures, nonce = stored + 1, before the deadline; tightening 1, loosening 2	Proven, Gate	`MandateHookConsortium.t.sol` (all 22 tests, incl. `test_signerSetRules`, `test_otherDomainRefused`, `test_rejectsMalleableHighS`, `test_deadlinePassedRefused`). `HookFuzz.t.sol`: `testFuzz_I3_signerSets`. `Anchor.t.sol`: `test_oneMemberIsNotEnough`, `test_nonMemberRefused`. FISCO `Board.t.sol`: `test_looseningNeedsTwoAndSignaturesAreSortedBySigner`, `test_duplicateSignerRefused`, `test_digestMatchesTheGoldenVector`. e2e F9 ("5: loosening to ACTIVE needs two members")
I4	No heartbeat before expiry means no autonomous action	Proven, Gate	`MandateHookDecide.t.sol`: `test_A8_staleHeartbeat`. `MandateHookConsortium.t.sol`: `test_heartbeatEpochAndTtl`. `services/operator/test/heartbeat.test.ts`. e2e F9 ("6: without heartbeats autonomous actions become STALE_STATUS; human approvals still work"). STALE_STATUS is a T2 reason: a human approval still lifts it, as spec 02 says
I5	A human approval is bound to one `callHash`, single use, expiring, with a fresh nonce	Proven, Gate	`MandateHookOwner.t.sol`: `test_approvalReplayAndSlotRules`, `test_approvalExpiryWindow`, `test_approvalRefusesWrongPasskeyOrUnknownCredential`, `test_ownerActionDigestSingleUseAndExpiry`. `MandateHookExecute.t.sol`: `test_approvalIsConsumedOnce`. `HookFuzz.t.sol`: `testFuzz_I5_approvalReplay`. `services/mcp/test/app.test.ts` ("refuses a call without a live, unconsumed approval for its callHash, and a batch": the resubmission after an approval signs only that call, D72). e2e F6
I6	The SOTER counter strictly increases per binding; the binding must match the account's	Proven, Gate	`MandateHookOwner.t.sol`: `test_soterCounterAndBinding`, `test_soterRefusesWrongDigestAccountExpiryOrInactiveAttester`. `HookFuzz.t.sol`: `testFuzz_I6_soterCounters`. `services/soter-attester/test/attester.test.ts` ("refuses a result Tencent does not verify, a raw that is not the digest, a bad code, a far expiry"). e2e F6 ("failure: a replayed SOTER counter is refused on-chain"), F2 ("failure: a SOTER result over another digest is refused by the attester"). Binding and counter are checked on-chain; the attester checks only that the SOTER result signs the exact digest
I7	The agent key can only send `execute`; never rules, keys or modules	Proven, Fork, Gate	`AgentKeyValidator.t.sol`: `test_refusesInstallAndUninstallModule`, `test_refusesExecuteTargetingTheAccount`, `test_refusesOtherCallTypes`, `test_onlyHookSetsAndRevokes`. `Safe7579Fork.t.sol`: `test_I7_agentCannotChangeModulesEvenWithAnApproval`. `services/agent-signer/test/signer.test.ts` ("refuses another sender, a non-execute call, a call to the account itself, another chain or entry point"). Integration `bundler-base.test.ts` ("I7: the agent key cannot change modules")

#	Invariant	Status	Proven by
I8	Effective limit = min(policy, bank caps); bank caps never exceeded, even with T2 or T3	Proven	`HookInvariants.t.sol`: `invariant_I8_bankCapsNeverExceeded`, `invariant_spentEqualsAllowedPayments`. `MandateHookDecide.t.sol`: `test_A5_overHardCapPolicyOrBank`, `test_A6_bankDailyCapIsT0EvenWithApproval`, `test_A6_missingCapsDenyByDefault`, `test_C6_bankMaxWeightIsT0`. `MandateHookExecute.t.sol`: `test_executorPathRefusesSwapsAndBankCaps`. e2e F3 ("failure: a hard cap above the bank cap of the profile is refused")
I9	KYC data never leaves the bank service; MCP and agent never receive PII	Proven, Gate	`services/bank/test/kyc.test.ts` ("round-trips and never contains the plaintext", the vendor-path test whose status "has no PII"). `kyc-routes.test.ts` ("answers 409 to a replayed decision and never echoes personal data"). Integration `bank-kyc.test.ts` ("the record at rest holds no plaintext personal data", "I9: no service log contains the seeded personal data", the KYC mock keeps no image). e2e F5 ("I9: no tool output carries KYC data"). KYC data is sealed with AES-256-GCM (`KYC_ENC_KEY`); MCP tools read no KYC data by construction
I10	Secrets never in git; TEE keys never leave the enclave	Proven, Partial	`test/infra/repo.test.ts` ("secrets stay out of git (I10)": `.env` and key files ignored, secret directories out of Docker build contexts, gitleaks in `make test`). `scripts/checks/gitleaks.sh`. agent-signer: no route returns key material (`signer.test.ts`, "never returns key material"); keys exist only through `getKey` inside the TEE. Partial: the TEE half is proven on the dstack simulator; on Phala it is a deployment check (tee-checklist)
I11	Refusal reasons in `RefusalLog` are computed by the hook's `check()`	Proven, Gate	`RefusalLog.t.sol`: `test_logsTheReasonTheHookComputes`, `test_refusesToLogAnAllowedCall`, `testFuzz_reasonAlwaysMatchesCheck`. Integration `bundler-base.test.ts` ("submits a refused op for evidence and logs the reason the hook computes (I11)"). e2e F5
I12	All arithmetic checked; overflow means refuse (`check()` never reverts)	Proven	`MandateHookDecide.t.sol`: `testFuzz_checkNeverReverts`, `testFuzz_checkNeverRevertsOnKnownTargets`, `testFuzz_checkBatchNeverReverts`, `test_A1_invalidAmount`, `test_C1_invalidAmount`. `MandateHookConsortium.t.sol`: `test_capsAbove2pow128Refused`. `packages/protocol/test/hash.test.ts` ("refuses amounts above 2^128 before any arithmetic", "refuses a price at or above 2^128 or more than 36 decimals"), D17
I13	Prod config refuses simulator attestations and debug-mode TEEs	Partial	`packages/config/test/config.test.ts` ("prod refuses the simulator, the fake Tencent, the fork, default passwords and an unthrottled sandbox", "only prod triggers the prod refusals"). `services/agent-signer/test/config.test.ts`, `services/soter-attester/test/config.test.ts`, `services/bank/test/flags.test.ts` (each "prod refuses simulator ..."). `services/bank/test/attestation.test.ts` ("phala mode: needs the verifier to accept and the compose hash on the allowlist"). `services/api/test/wechat.test.ts` ("prod refuses the fake and any WeChat base but the official one"). `test/infra/compose-prod.test.ts` ("never starts the scripted model", "has a rehearsal target that runs the smoke suite and the prod refusals"); `scripts/deploy/prod-refusals.sh` checks the five services' refusals in the production compose (`make rehearsal`, passed in the M18 rehearsal, D90). Not yet: the debug-mode refusal relies on Phala's verifier and is checked at deployment (Phase C); see the per-service gaps in section 5
I14	While FROZEN, nothing moves value out (payments, swaps, payouts, standing orders)	Proven, Gate	`HookInvariants.t.sol`: `invariant_I14_nothingLeavesWhileFrozen`. `MandateHookDecide.t.sol`: `test_rule1_frozenWinsOverEverything`, `test_approvalNeverLiftsFrozen`. `MandateHookExecute.t.sol`: `test_frozenBlocksModuleChanges` (D19). `StandingOrders.t.sol`: `test_hookStillRefusesFrozenSanctionsJurisdictionPartnerAndCaps`. e2e F9 ("3: the next agent action is refused FROZEN on-chain (I14)")
I15	A rebalance never ends with an asset outside its band unless it moved closer to target	Proven, Gate	`HookInvariants.t.sol`: `invariant_I15_bands`. `HookFuzz.t.sol`: `testFuzz_I15_projectionMatchesOutcome`. `MandateHookExecute.t.sol`: `test_postCheckEnforcesBandsOnRealBalances`, `test_postCheckCatchesABreachTheProjectionMissed`, `test_postCheckSkipsTheBandRuleWhenAHumanApprovedTheBreach`. `MandateHookDecide.t.sol`: `test_C10_bandBreachNeedsAHuman`. e2e F4

#	Invariant	Status	Proven by
I16	No signature path outside the hook: no valid ERC-1271 signature	Fork, Proven	`Safe7579Fork.t.sol`: `test_I16_usdcPermitAndAuthorizationWithAccountSignaturesFail`. `AgentKeyValidator.t.sol` and `PasskeyValidator.t.sol`: `testFuzz_erc1271AlwaysInvalid`. `NoOwner.t.sol`: `testFuzz_neverSigns`. The hook also refuses USDC `permit`, `transferWithAuthorization`, `receiveWithAuthorization` and `transferFrom` (D10, `test_usdcSelectorsOutsideThePatternRefused`)
I17	Every non-view selector of the Safe7579 adapter is hook-wrapped or unreachable	Fork	`Safe7579Fork.t.sol`: `test_I17_outsidersCannotReachAnyNonViewEntryPoint`, `test_I17_ourValidatorsRefuseEveryOtherSelector`
I18	A consortium message cannot block future messages (nonce = stored + 1)	Proven	`MandateHookConsortium.t.sol`: `test_nonceMustBeStoredPlusOne`. `HookFuzz.t.sol`: `testFuzz_I18_nonce`. `Anchor.t.sol`: `test_sequenceAndDeadline`. FISCO `Board.t.sol`: `test_openAllocatesIdNonceAndFiscoRef`, `test_reissueAnUndeliveredExpiredMessageWithTheSameNonce`, `test_reissueRefusedWhenDeliveredNotExpiredOrSuperseded` (D25)
I19	No personal data on Base: only addresses, amounts and hashes	**Not yet** (by design only)	No test scans Base calldata and events yet: the scan of the Base data (the live run's included) is not built and is left to Phase C. By design: payees are addresses, merchants are `merchantRef = keccak256(merchantId)`, payout references are hashes, labels live in the api and mcp databases. As built, Base also carries each account's KYA attributes (kycLevel, riskProfile, a two-letter jurisdiction) and the ERC-3643 identity country, as spec 02 and 03 define them (D69)

4. Threat model (spec 08, updated to the build)

Attacker	Can	Controls as built	Gaps as built
Prompt injection (merchant text, chat, payee name, tool result)	Make the LLM call tools with bad arguments	No MCP tool takes an account (the account comes from the bearer); the hook's default deny, sanctions, caps and payee list; anything unusual is T2; OpenCode's built-in tools are denied (not offered) or rejected by agent-runner (D15, D86, D88). Tests: e2e F5 ("I9", "the built-in tools are locked"), `services/agent-runner/test/runner.test.ts`. Red team item 1 on 2026-10-18: 12 prompt-injection cases, run scripted and with the real model, 0 executed (section 9)	Held-out red-team cases not run yet (MANUAL 7.1)
Compromised backend (api, scheduler, bundler)	Withhold, delay, spam, show the user wrong details	Every authority is a signature verified on-chain; refused ops are rate-limited (20 per account per day, `BUNDLER_MAX_REFUSED_PER_DAY`); the bundler sends plain transactions only to five allowlisted contracts; member services verify staff passkey assertions themselves (D24); the heartbeat fail-safe. What you see is what you sign: the web app recomputes the Tier2Approval digest and every owner action's digest, kind, account and payload hash before the passkey signs (D46; `apps/web/src/lib/web.test.ts`: "signs an owner action only when its digest, kind, account and payload hash are what the page asked for", "signs a policy only when the payload is the policy on the page"); the mini program recomputes the call hash and the digest before SOTER signs and refuses another owner action shown as a link (D52; `apps/miniprogram/test/verify.test.mjs`: "refuses a digest that does not match the shown request", "refuses another owner action shown as a link (e.g. CANCEL_RECOVERY, which SOTER can sign)"); mcp resubmits only a call whose approval is live on the hook (D72); the web app reads a new device's key from the browser's own registration and signs an AgentMandate draft only for the agent key it shows (D83; `apps/web/src/lib/devices.test.ts`, `apps/web/src/lib/agent.test.ts`)	The WeChat link confirmation, the deployment userOp and console proposals are signed as the api returns them (risk 22); the api carries the recovery passkey to the bank and relays the recovery status (risk 31); agent-runner's `INTERNAL_TOKEN_MCP` mints an agent bearer for any account (risk 19)

Attacker	Can	Controls as built	Gaps as built
Compromised mcp (the only caller of `sign-userop`)	Act inside the T1 mandate, like a leaked agent key	The hook's bounds and the bank caps; agent-signer limits `sign-userop` to 120 per account per hour and refuses anything but `execute` to other targets on chain 84532; owner revoke (T3-); consortium freeze	The signer's rate limit is in memory (reset on restart)
Compromised WeCross router or relayer	Drop or delay consortium messages	Member signatures, nonces and deadlines verified by the hook; silence makes the agent RESTRICTED (STALE_STATUS); the relayer's watchdog delivers what WeCross did not within `BRIDGE_WATCHDOG_S` (D42)	WeCross 1.4.0 notes in wecross.md (risk 18)
Leaked agent key or compromised agent TEE	Act inside the mandate	Hook bounds and bank caps; the owner revokes alone; the consortium freezes; the owner rotates to the next key version with a new KYA, after which the old key no longer validates and the bank revokes its AgentMandate (F10, D47; `services/api/test/routes.test.ts`: "asks the bank to revoke the mandates of agent keys Base dropped, after a revoke or a new key lands"; mcp asks agent-signer for the key Base holds, `services/mcp/test/ports.test.ts`: "asks the signer for the key that is the account's agent key on-chain (F10: after a rotation)")	Risk 1. A rotation stays inside the same TEE: a compromised enclave derives every version
Recovery abuse (someone who knows the owner's email and identity data)	Have the owner's passkeys replaced	Only the bank schedules a recovery, after its own re-KYC with the provider matches the sealed PersonKYC record (name, birth date, nationality, document number); the account is RESTRICTED for 48 h and any remaining passkey cancels alone; the new passkey signs in only once Base holds it (`devices.test.ts`, `recovery.test.ts`, e2e F10 step 6)	Risk 31
Compromised attester key	Sign SOTER approvals for every SOTER-enabled account, T2 payments up to each hard cap	Bank caps stay T0; approvals expire within one hour (`MAX_TTL`); any one member deactivates the attester (`test_attesterActivateBankDeactivateAny`); SOTER is off unless the policy sets `soterEnabled`	Risk 14
False Tencent `is_ok` alone	Nothing without a SOTER signature from the bound phone over the exact digest	The attester refuses a result whose `raw` is not the digest (`attester.test.ts`); the hook checks the binding and the counter	Risk 2
Stolen phone	Approve as the user	Phone biometrics; the owner unlinks WeChat alone (T3-); the bank can freeze through the consortium	
One malicious member key	Freeze or restrict (the safe direction), list sanctions, suspend an asset	Loosening needs two members; every decision is a FISCO proposal with its signers in the audit trail	
Replay of old messages or approvals	Re-apply an old status or approval	Nonces (stored + 1), deadlines, single-use digests (`usedDigest`), SOTER counters, epochs reset by recovery	
Oracle feeder compromise	Push wrong prices	At most 25 % per update (`MAX_DEVIATION_BPS`), staleness (`maxPriceAge`, 4 days), slippage bound, the bank's max weight (T0), DEMO prices flagged	Risk 6

Attacker	Can	Controls as built	Gaps as built
Web attacker	XSS, CSRF, session theft, OAuth abuse on MCP	`HttpOnly`, `SameSite=Lax` session cookie (`Secure` on https); CSRF token on every state-changing api request; OAuth 2.1 with PKCE S256 only, exact redirect URIs, single-use codes, refresh rotation with family revocation, tokens stored as hashes (`services/mcp/test/oauth.test.ts`, 28 cases); web push messages are encrypted for each browser (RFC 8291) and signed with Praetor's VAPID key (RFC 8292), to https endpoints only (`services/notifier/test/webpush.test.ts`: "posts an aes128gcm-encrypted payload with a VAPID authorization to every subscription of the account"; `services/api/test/routes.test.ts`: "refuses a subscription whose endpoint is not https")	Since D71: a strict CSP (scripts from self only, no framing, no objects), HSTS, nosniff and no-referrer at Caddy; writes from another Origin refused by the api; per-client rate limits on sign-in, onboarding, recovery and mini program sign-in (`services/api/test/hardening.test.ts`, `test/infra/caddy.test.ts`). Since D82: web push only to browser push services, at most ten per account, no redirect followed (`packages/protocol/test/push.test.ts`, `services/notifier/test/webpush.test.ts`), notification links kept on the app's origin (`apps/web/src/lib/sw.test.ts`), no route can be framed. Remaining: risk 21
Insider reading logs	Learn PII	KYC data sealed in the bank database; status responses carry no PII; the bundler and the member services redact the authorization header in their logs; I9 log scans in the integration suite	The api stores the sign-up email (not KYC data)

5. Known risks

Items 1 to 17 are spec 08's list, updated to the build; items 18 and later were added during the build, found in M17, in the docs refresh (D85) or in the 2026-10-18 security review (D80 to D83).

1. **Agent key.** A compromised agent TEE or leaked agent key can act inside the mandate until the owner revokes or rotates it (F10) or the consortium freezes the account. Bounded by the hook and the bank caps.
2. **SOTER trust.** SOTER approvals trust Tencent's unsigned `is_ok` and the attester's code. Bounded by the binding, the counter, the one-hour expiry, the hard cap and the bank caps.
3. **One team runs the demo consortium.** In production bank, regulator and operator are independent organisations with their own HSMs.
4. **Unbacked tokens.** tXAU and tXAG are demo tokens; production needs a regulated custodian with proof of reserves.
5. **Mocked payouts.** The payout partner and the Alipay+ and WeChat Pay gateways are labelled mocks (the payout partner runs the real `ant-intl-alipayplus-mcp` client against the mock gateway). Production needs a licensed PSP; stablecoin payouts only where licensed, never for mainland China residents (`JURISDICTION_BLOCKED`).
6. **Oracle feeder.** A single key, bounded to 25 % per update. It reads Pyth Hermes only when `PYTH_API_KEY` is set and `CHAIN_MODE=live` (one request every 5 minutes, D89); on the fork, or without the key, it pushes seeded prices flagged DEMO (D13, BLOCKERS B2), and no service refuses DEMO mode in prod.
7. **RefusalLog link.** The link from a `Refused` record to its userOpHash is the reporter's claim; the reason is not. The bundler writes RefusalLog only for an op refused in its pre-check that then failed on-chain.
8. **WeCross.** WeCross 1.4.0 is unmaintained upstream; the EVM stub is new code (`wecross/evm-stub`, JUnit vectors checked against viem).
9. **Freeze blocks the owner.** A consortium freeze also blocks the owner's own payments and module changes (D19): a regulated account by design.

10. **Mini program.** The mini program runs on a test account in debug mode; publication needs a registered company and ICP filing. Built in M14. Tested by the team on 2026-10-18 on an iPhone with the WeChat test account, against the local fork through `make tunnel` (MANUAL 5): the WeChat link, a SOTER Face ID approval with the same digest on the phone and the web app, a replayed approval refused on-chain (DigestUsed), the regulator freeze (6.3 s from FISCO to Base; loosening 7.5 s), the unlink and the Chinese UI passed. Not tested: the Android fingerprint (WeChat refused a second phone); web push on a phone and the sandbox page were not tested by a person.
11. **30-day epochs.** Monthly budgets use `block.timestamp / 30 days`, not calendar months: up to two monthly budgets across a boundary.
12. **ERC-3643 agent role.** The bank's token agent key can force-transfer or burn tXAU and tXAG of any holder (the standard's recovery power), outside the hook (`Tokens.t.sol`, `test_agentForcedTransferBurnAndRecovery`).
13. **Compromised mcp.** mcp is the only service allowed to call `sign-userop`; compromised, it acts inside the T1 mandate. See also item 19.
14. **Compromised attester key.** It can approve T2 payments for every SOTER-enabled account until any one member deactivates it.
15. **Sealers.** In the demo the operator runs 2 of the 4 FISCO sealers and can halt the consortium chain; production gives each member its own sealer and adds a fourth member.
16. **SOTER counter on iOS.** If the counter does not increase on iOS, the hook's `soterCounterStrict` (an immutable constructor flag, true by default through `SOTER_COUNTER_STRICT=1`) must be deployed as false; replay stays impossible because each approval's digest is single use (on the team's iPhone a replayed approval was refused on-chain with DigestUsed, risk 10).
17. **WeIdentity REST** is custodial and unauthenticated; it runs on its own `identity` network joined only by the bank, MySQL and one FISCO node (D29, `test/infra/compose.test.ts`).
18. **WeCross transport notes** (wecross.md): one undeliverable request can stall the hub queue (the watchdog does not depend on it); two hub setters have no access control; on a failure the router logs a chain account's private key (kept on the host or CVM only); the router RPC is plain HTTP on the Docker network; the bridge-relayer holds the WeCross router's FISCO key in `BRIDGE=wecross` mode (D42).
19. **Internal tokens that reach the agent key.** mcp's `POST /internal/resubmit` (used after a human approval) is behind `INTERNAL_TOKEN_API`, which the api, the notifier and mcp share. Since D72 it signs only a single call whose approval is live and unconsumed on the hook (`services/mcp/test/app.test.ts`), so that token reaches the agent key only for calls the owner approved. agent-runner holds `INTERNAL_TOKEN_MCP`, which mints an agent bearer for any account: a compromised agent-runner can act inside the T1 mandate of every account, like a compromised mcp. The hook still bounds every call.
20. **Shared internal tokens.** `INTERNAL_TOKEN_MEMBERS` is shared by the api and the three member services, `INTERNAL_TOKEN_BUNDLER` by every bundler caller, `SESSION_SECRET` by the api and mcp. Consortium signatures still need a staff passkey assertion over the exact digest, except four automated, rule-checked signatures: the operator's heartbeat, the bank's KYA after its own checks, the bank's owner-action co-signature after its rule checks, and the bank's anchor co-signature after recomputing the chain (D24, D26, D41). F10 adds a fifth automated bank signature outside the consortium boards: the `SET_OWNER_KEY` of a recovery, signed and sent (`scheduleRecovery`) only after the bank's re-KYC matched the sealed record (D47; bounded by the 48 h delay, the RESTRICTED status meanwhile and the owner's cancel, risk 31).
21. **Web hardening, as built.** Done since D70 and D71: CSP and security headers at Caddy, an Origin check on the api's state-changing requests, per-client rate limits on the public auth routes, and the console's account list behind a staff session. Since D82: push subscriptions limited to browser push services and ten per account, no redirect followed by the notifier, notification clicks kept on the app's origin, `X-Frame-Options: DENY` on every route (`packages/protocol/test/push.test.ts`, `services/notifier/test/webpush.test.ts`, `apps/web/src/lib/sw.test.ts`). Remaining: no rate limit in the member services or the KYC mock (internal

network only); the limits are per process (in memory) and keyed on the address Caddy sees, which is one gateway address for everyone behind `make tunnel` or a TLS-passthrough gateway (the sandbox faucet's per-client limit included); the api's pending WebAuthn challenges are never swept, only bounded by the rate limits.

22. **What you see is what you sign, partly.** The web app recomputes the Tier2Approval digest from the request's preimage before the passkey signs; since D46 it also recomputes every owner action's digest and checks its kind, account and payload hash, and rebuilds the payload from what it shows for a policy, an agent key, a new passkey, a revocation and a recovery cancellation. Until D83 the new passkey it showed was the api's answer, so a compromised api could have had the owner sign SET_OWNER_KEY for the api's own key (the bank co-signs that action without a key check); the page now reads the key from the browser's own registration and refuses another (`apps/web/src/lib/devices.test.ts`), and it signs an AgentMandate draft only for the agent key it shows (`apps/web/src/lib/agent.test.ts`). Still signed as the api returns them: the WeChat link confirmation, the account's deployment userOp and console proposals (the member service verifies the staff assertion over the proposal digest it reads from FISCO itself).
23. **Staff WebAuthn details.** Member services accept high-s P-256 staff signatures and do not check the signature counter. Malleability is not forgery and challenges are single use; low impact.
24. **In-memory state.** Staff sessions and challenges, the bank's quote nonces, the WeChat link sessions, agent-signer's rate limit, the api's rotated agent key between `/agent/rotate` and the mandate draft, and the sandbox faucet's funded accounts and per-client counts live in memory and reset on restart.
25. **Prod refusals are per service.** `packages/config` refuses simulator attestations, the fake Tencent, the fork, default passwords and an unthrottled sandbox, but only the bundler, the bridge-relayer and the member services load it; agent-signer, soter-attester, the bank, agent-runner and the api carry their own checks (simulator, fake Tencent, WeChat API base, fake LLM; the api refuses the fake Tencent and any other WeChat API base, `services/api/test/wechat.test.ts`); oracle-feeder's DEMO mode is not refused. The sandbox faucet is in the api, which does not load `packages/config`: it limits funding to `SANDBOX_RATE_LIMIT_PER_HOUR` per client (3 by default) and mints only on the fork, so it refuses on live Base (`services/api/test/sandbox.test.ts`: "limits funding per client address per hour", "exists only when the sandbox is enabled, and mints only on the fork"), once per account; but anyone can create accounts (60 sign-ups a minute per client), so a patient visitor can still mint fork USDC and buy out the fork desk's tXAU and tXAG. `scripts/deploy/prod-refusals.sh` checks five services' refusals in the production compose (passed in the M18 rehearsal, D90).
26. **KYC paths.** The EUDI path trusts the EU verifier backend for the credential signature and yields kycLevel 1 without name screening (D30); the vendor HMAC covers the body but no timestamp (replay is stopped by the session state machine); the KYC vendor is a labelled mock whose decision is always "approved" (the bank still checks liveness, age, sanctions and jurisdiction). The recovery re-KYC compares the typed name, birth date, nationality and document number with the sealed record (not the photos), so with the mock anyone who knows those four fields can schedule a recovery: the owner is told at once (D81) and any remaining passkey cancels within 48 h; scheduling again after each cancel keeps the account RESTRICTED. A user verified through EUDI only cannot recover (the re-KYC is the vendor's).
27. **Pseudonymous attributes on Base.** KYA attestations put kycLevel, riskProfile and a two-letter jurisdiction per account on Base, and the ERC-3643 identity registry stores a country code (D69).
28. **USDC is implicitly approved** (D21): the consortium cannot delist USDC itself; Circle's own controls apply.
29. **Process.** `make slither` passes since D49 because it filters the vendored WeCross proxy like `lib/`; the proxy's findings are triaged one by one in slither.md, not gated. The gitleaks worktree caveat found in M17 is fixed (the script mounts the common git directory). Dependency advisories of deps.md are not upgraded yet.
30. **Test-only services in the default profiles.** The scripted fake LLM starts with the `agent` profile; agent-runner refuses `FAKE_LLM_URL` when `PRAETOR_ENV=prod`, so a prod deployment must leave it unset. `infra/compose.prod.yml` moves the fake LLM to a `test` profile, so the production compose never starts it.
31. **Recovery, as built (F10).** The KYC vendor is a labelled mock that always approves and the re-KYC compares typed fields (risk 26), so in the demo anyone who knows an owner's email and the four identity fields of the sealed record can schedule a recovery; the owner's protection is the 48 h delay with the account RESTRICTED and any remaining passkey able to cancel. The bank's recovery notice goes to its own log sink (D13); since D81

the notifier also pushes every `RecoveryScheduled` to the owner's devices with the time the keys go and how to cancel (`services/notifier/test/notifier.test.ts`), and the owner sees the pending recovery in the web app's Settings and through the RESTRICTED status. A stranger who knows the email can add at most five pending recovery passkeys an hour to its user (D80); they never count toward the device limit. Recovery needs a PersonKYC issued from the KYC provider: an owner onboarded with the EUDI wallet cannot recover (the bank refuses). After execution the old approvals are void (new epoch), the WeChat binding is cleared and the agent key is revoked, so the owner installs a new agent key (`SET_AGENT_KEY`, owner and bank); a mini program session opened before the recovery ends at its next request, once Base no longer holds its binding (D81, `services/api/test/mp-session.test.ts`). Through a compromised api: the api is the only path of the recovery passkey to the bank and of the recovery status to the web app. A compromised api can send the bank its own key while the user does the re-KYC, and hide the pending recovery from the Settings page; a user who lost every passkey cannot cancel. Likewise, a recovery's KYC link is a bearer capability and the provider's page does not say it is a recovery: whoever starts a recovery for a victim's email can send the victim that link, and a victim who completes the check schedules the stranger's key. Bounded by the 48 h delay, the notifier's push (a separate service), the bank's own notice, and a freeze by any one member.

32. **Email as the recovery handle.** Sign-up emails are not verified. Recovery resolves an email to its first account (D80), so whoever signs up with someone's email before they do holds that recovery entry point. `/api/recover/options` tells whether an email has an account (404 or 200, rate-limited per client, D71) and returns its user id, with which `/api/recover/:userId` shows the recovery state.
33. **Sessions outlive keys.** The web session is a stateless 12 h cookie: it survives a recovery and a lost or replaced passkey until it expires. Authority still needs a passkey assertion or the agent's mandate, and a recovery revokes the agent key. Mini program sessions end at once since D81.
34. **`make tunnel`.** The quick tunnel exposes the local stack (fork, fake Tencent, KYC mock) to anyone who has its trycloudflare URL: stop it after the phone tests (`make tunnel-stop`). In prod the Caddyfile's plain-HTTP tunnel site answers a client that sends its placeholder host name; it serves the same app over HTTP, where the Secure cookie and passkeys do not work.

6. Static analysis

Full report: slither.md. Praetor's Base contracts: **High 0, Medium 27, Low 52**, unchanged since M2; every Medium is explained (zero-initialised locals, deliberately ignored return values, non-balance equalities). The vendored WeCross proxy (M12) adds High 3, Medium 8, Low 11, triaged one by one as not exploitable: the proxy holds no authority and every message it carries is verified by the hook. It made `make slither` exit with code 255 at M17 (D66); since D49 the gate filters `src/vendor/` like `lib/` and passes, the triage staying in slither.md. FISCO contracts, now in the same gate: High 0, Medium 5 (the same explained classes), Low 13. `forge lint` was not run.

7. Coverage

`FOUNDRY_PROFILE=quick forge coverage --no-match-path 'test/fork/*' --report summary` in `contracts/base` (fork tests excluded because they need the network; forge disables the optimizer and `via_ir` for coverage, so the vendored proxy, which needs `via_ir`, has no row). Target: 90 % of lines on MandateHook (spec 08).

Source	Lines	Statements	Branches	Functions
`MandateHook.sol`	**99.52 % (207/208)**	99.59 % (242/243)	100 % (38/38)	100 % (53/53)
`hook/DecisionLib.sol`	99.66 % (295/296)	96.41 % (510/529)	82.83 % (82/99)	100 % (31/31)
`hook/OwnerLib.sol`	99.43 % (175/176)	95.14 % (274/288)	81.54 % (53/65)	100 % (15/15)

Source	Lines	Statements	Branches	Functions
`hook/ConsortiumLib.sol`	100 % (97/97)	97.76 % (131/134)	80.00 % (12/15)	100 % (13/13)
`hook/PostCheckLib.sol`	97.92 % (47/48)	97.83 % (90/92)	86.67 % (13/15)	100 % (5/5)
Hook with its libraries	**99.52 % (821/825)**			
`AgentKeyValidator.sol`	100 % (44/44)	98.18 %	88.89 %	100 %
`PasskeyValidator.sol`	93.65 % (59/63)	93.51 %	93.33 %	87.50 %
`RefusalLog.sol`, `Anchor.sol`, `NoOwner.sol`, `PayoutRail.sol`, `PraetorPaymaster.sol`, `PriceOracle.sol`, `TreasuryDesk.sol`, `WeCrossInbox.sol`	100 % each			
`StandingOrders.sol`	97.06 % (33/34)	100 %	100 %	87.50 %
`tokens/PraetorToken.sol`	95.52 % (64/67)	91.55 %	71.43 %	94.12 %
`tokens/IdentityRegistry.sol`	85.00 % (17/20)	84.21 %	50.00 %	87.50 %

The target is met. Branch coverage of the hook's libraries (80 % to 87 %) is lower than line coverage; the uncovered branches were not reviewed one by one for this document.

8. Dependencies, secrets and licences

- **Dependency audit** (deps.md): `pnpm audit --prod` reports 4 high, 7 moderate and 2 low advisories. Not reachable as used: the MCP SDK OAuth client (we use stdio), drizzle identifiers (static), body-parser limits, react-router (no SSR, fixed routes), uuid. Low: mysql2 (needs a hostile MySQL server on the internal network). To upgrade: express's qs and path-to-regexp (DoS on public routes). OWASP dependency-check for the Java parts and image scanning are open items.
- **Secrets**: `.env` (mode 600) is git-ignored, keys are append-only (`pnpm keys`), services never log values; gitleaks runs in `make test` and found no leak in the history of this branch at this commit.
- **GPL boundary** (THIRD_PARTY_LICENSES.md): no GPL or LGPL source in `contracts/` or `packages/` (every Solidity file is `UNLICENSED` except the Apache-2.0 vendored proxy); the 51 sources compiled into the Base contracts are 26 `UNLICENSED`, 24 MIT, 1 Apache-2.0. GPL contracts (Safe7579, launchpad, registry, EntryPoint) and LGPL Safe are only deployed and called through our own interfaces.

9. Red-team results (M16)

The suite (`test/redteam`, the attack classes of spec 08) ran against the full local stack (fork, domain `Praetor-local`) on 2026-10-18 and wrote redteam.md: each case with its input, expected and observed outcome and transaction hashes, reported as "N attacks, 0 executed".

Run	Command	Attacks	Executed	Outcome
Scripted	`make redteam`	85	**0**	85 blocked where expected
Global (shared state, restored afterwards)	`make redteam-global` (`REDTEAM_GLOBAL=1`)	3	**0**	3 blocked: a swap into a DELISTED tXAG, a swap while every price is stale, a payment during a global freeze
Live, real model (DeepSeek through OpenCode)	`make redteam-live`	12	**0**	3 blocked by the stack, 9 declined by the model; their scripted twins show the stack blocks the same attacks

Executed (D60) means a value or a permission moved that the mandate does not allow, judged on-chain before and after each attack (token transfers out of the account, policy hash, agent key, binding, status and its nonce, the hook, owner keys, approvals and owner actions), or a forbidden request the stack accepted (an attack userOp or transaction that succeeded, HTTP 2xx or 101, an OAuth token or redirect issued, seeded KYC data in a tool output); never what a tool or the model said. The scripted run passes only when every case is blocked, the live run when each is blocked or declined by the model, and an error always fails. The live driver counts only outflows outside the mandate, since the real model may pay an allowlisted payee within it on its own.

Findings of the earlier runs (D62), none of which moved value, all fixed and re-tested:

- **D86.** The web tools are denied in each account's OpenCode configuration, so they are not offered (a `webfetch` call hung the turn); a turn that times out aborts its OpenCode session; Caddy is recreated when its file changes, so the Content-Security-Policy, HSTS and X-Frame-Options headers are served (RT-82, RT-84).
- **D87.** mcp takes an agent bearer only from agent-runner's address (the Docker host's bridge gateway was admitted before, RT-73); a payee name resolves to the exact label, else to the only label containing it; one FISCO or desk transaction in flight per sender; the demo account keeps the real model.
- **D88.** Six OpenCode built-in tools are denied, so not offered: `webfetch`, `websearch`, `glob` and `grep`, whose pending permission request breaks OpenCode's permission list (the runner saw nothing to reject and the turn hung), plus `lsp` and `question`; the other built-in tools stay rejected by agent-runner. A turn whose pending permissions cannot be listed stops at once and aborts its session (fail closed; RT-85 is the scripted twin of what the real model did).

Held-out cases (written outside the repo) use the suite's JSON case schema and run through `REDTEAM_HELDOUT_DIR` (D64); none were part of these runs (MANUAL.md step 7.1).

10. Pentest (Phase C)

To be filled in Phase C, with the team's AI pentest pipeline and manual testing: web app, api, mcp and OAuth, mini program backend, WeChat callback, console. Report: `docs/security/pentest.md`, with findings and fixes. Starting points from this assessment: risks 19 to 25 and 31.

11. TEE checklist results (after deployment)

The checklist and what the code enforces today: tee-checklist.md. Results per CVM (agent-signer, soter-attester) **after the Phala deployment in Phase C**.